

Contents

□ DSA Pattern Mastery — Master Roadmap	1
□ Folder Structure	1
□ Why This Order?	2
□ The 28 Patterns, In Learning Order	2
□ How To Use This Series	5
□ Progress Tracker	5

□ **DSA Pattern Mastery — Master Roadmap**

Target audience: Senior/Staff Software Engineer interview prep (8-12 YOE) — Google, Amazon, Meta, Microsoft, Uber, Airbnb, Atlassian, Grab, Careem, Noon, Talabat, Amazon UAE, Dubai/Saudi product companies, India Tier-1/Tier-2 product companies.

Companion resource: Designed to be studied alongside **Striver's A2Z DSA Course / SDE Sheet** video playlist. Each handbook below assumes zero external lookup — everything you need is inside that pattern's own document.

□ **Folder Structure**

Each pattern lives in its own numbered folder containing **both** a Markdown handbook and a PDF handbook (identical content):

```
DSA Patterns/
  00-Pattern-Roadmap.md           ← you are here
  01-Two-Pointers/
    Two-Pointers-Handbook.md
    Two-Pointers-Handbook.pdf
  02-Sliding-Window/
    Sliding-Window-Handbook.md
    Sliding-Window-Handbook.pdf
  03-Fast-Slow-Pointers/
  04-Linked-List-Reversal/
  05-Prefix-Sum/
  06-Binary-Search/
  07-Sorting-Cyclic-Sort/
  08-Merge-Intervals/
  09-Hashing/
  10-Monotonic-Stack/
  11-Monotonic-Queue-Deque/
  12-Recursion-Backtracking/
  13-Tree-DFS/
  14-Tree-BFS/
  15-Binary-Search-Trees/
  16-Trie/
  17-Heap-Priority-Queue/
  18-Graph-BFS-DFS/
```

19-Union-Find/
 20-Topological-Sort/
 21-Shortest-Path/
 22-DP-1D/
 23-DP-2D-Grid/
 24-DP-Knapsack/
 25-DP-Strings/
 26-DP-Bitmask/
 27-Greedy/
 28-Bit-Manipulation/

□ Why This Order?

Patterns are sequenced so every new pattern either **reuses a data structure you just mastered** or **builds directly on the previous pattern’s mental model**. This mirrors how Striver’s course layers concepts and how real interview prep should compound.

Arrays & Two Pointers Foundations

Windows & Prefix Techniques

Search Space Reduction

Linked List Pointer Tricks

Sorting-Based Tricks

Hashing & Stacks/Queues

Recursion Foundations

Trees (DFS/BFS/BST/Trie)

Heaps

Graph Foundations (BFS/DFS)

Graph Structure (Union-Find, Topo Sort)

Graph Weights (Shortest Path)

Dynamic Programming (1D → 2D → Knapsack → Strings → Bitmask)

Greedy

Bit Manipulation

□ The 28 Patterns, In Learning Order

#	Pattern	Core Idea	Typical Difficulty Curve	Folder
1	Two Pointers	Two indices moving toward/away from each other over a sorted or linear structure	Easy → Medium	01-Two-Pointers

#	Pattern	Core Idea	Typical Difficulty Curve	Folder
2	Sliding Window	A contiguous, dynamically resizing subarray/substring window	Easy → Hard	02-Sliding-Window
3	Fast & Slow Pointers	Two pointers moving at different speeds to detect cycles/midpoints	Easy → Medium	03-Fast-Slow-Pointers
4	In-Place Linked List Reversal	Rewiring next pointers without extra space	Easy → Hard	04-Linked-List-Reversal
5	Prefix Sum & Difference Array	Precomputed running aggregates for O(1) range queries/updates	Easy → Medium	05-Prefix-Sum
6	Binary Search on Answer	Searching a monotonic answer space instead of an array	Medium → Hard	06-Binary-Search
7	Sorting Techniques & Cyclic Sort	Using order (or index-as-hash) to simplify problems	Easy → Medium	07-Sorting-Cyclic-Sort
8	Merge Intervals	Detecting/merging overlapping ranges	Medium	08-Merge-Intervals
9	Hashing Patterns	O(1) lookup/frequency tricks	Easy → Medium	09-Hashing
10	Monotonic Stack	Maintaining increasing/decreasing order to answer “next greater/smaller”	Medium → Hard	10-Monotonic-Stack
11	Monotonic Queue / Deque	Sliding-window extremes in O(n)	Medium → Hard	11-Monotonic-Queue-Deque

#	Pattern	Core Idea	Typical Difficulty Curve	Folder
12	Recursion & Backtracking	Exhaustive search with pruning	Medium → Hard	12-Recursion-Backtracking
13	Tree DFS Patterns	Pre/in/post-order recursive tree traversal	Easy → Hard	13-Tree-DFS
14	Tree BFS Patterns	Level-order traversal via queue	Easy → Medium	14-Tree-BFS
15	Binary Search Trees	Order-preserving tree operations	Medium	15-Binary-Search-Trees
16	Trie (Prefix Tree)	Character-by-character prefix indexing	Medium → Hard	16-Trie
17	Heap / Priority Queue	Efficient min/max retrieval, Top-K, K-way merge	Medium → Hard	17-Heap-Priority-Queue
18	Graph BFS & DFS	Traversal fundamentals for connectivity/shortest unweighted path	Medium	18-Graph-BFS-DFS
19	Union-Find (DSU)	Efficient dynamic connectivity	Medium → Hard	19-Union-Find
20	Topological Sort	Ordering under dependency constraints (DAGs)	Medium → Hard	20-Topological-Sort
21	Shortest Path Algorithms	Dijkstra / Bellman-Ford / Floyd-Warshall	Hard	21-Shortest-Path
22	Dynamic Programming — 1D	State reduces to a single index	Medium → Hard	22-DP-1D
23	Dynamic Programming — 2D/Grid	State depends on two indices/coordinates	Medium → Hard	23-DP-2D-Grid

#	Pattern	Core Idea	Typical Difficulty Curve	Folder
24	Dynamic Programming — Knapsack	Choice-based sub-set/capacity optimization	Medium → Hard	24-DP-Knapsack
25	Dynamic Programming — Strings	LCS, edit distance, palindrome family	Hard	25-DP-Strings
26	Dynamic Programming — Bitmask	State compressed into a bitmask over small N	Hard → Very Hard	26-DP-Bitmask
27	Greedy Algorithms	Locally optimal choices with exchange-argument proofs	Medium → Hard	27-Greedy
28	Bit Manipulation	Manipulating binary representation directly	Easy → Hard	28-Bit-Manipulation

□ How To Use This Series

1. Study one pattern folder at a time, in order — do not skip ahead.
2. Read **Sections 1-5** first (Overview → Algorithm Framework) before touching any code.
3. Watch the corresponding Striver video for the pattern, then re-read **Section 4 (Why It Works)** to confirm your intuition matches the proof.
4. Attempt the **Practice Roadmap** (Section 23) problems in order: Beginner → Intermediate → Advanced → Expert.
5. The day before an interview, only re-read **Section 21 (Cheat Sheet)** and **Section 22 (Revision Notes)** of every pattern you've covered.

□ **Note:** Each handbook is self-contained and written assuming zero external lookup — treat each one as the single source of truth for that pattern.

□ Progress Tracker

Pattern #	Status
1-28	☐ Not started → will be updated to ☑ as each handbook is completed in this folder

(This roadmap file is the master index. Individual completion status is tracked in the session's task list while the series is being generated.)